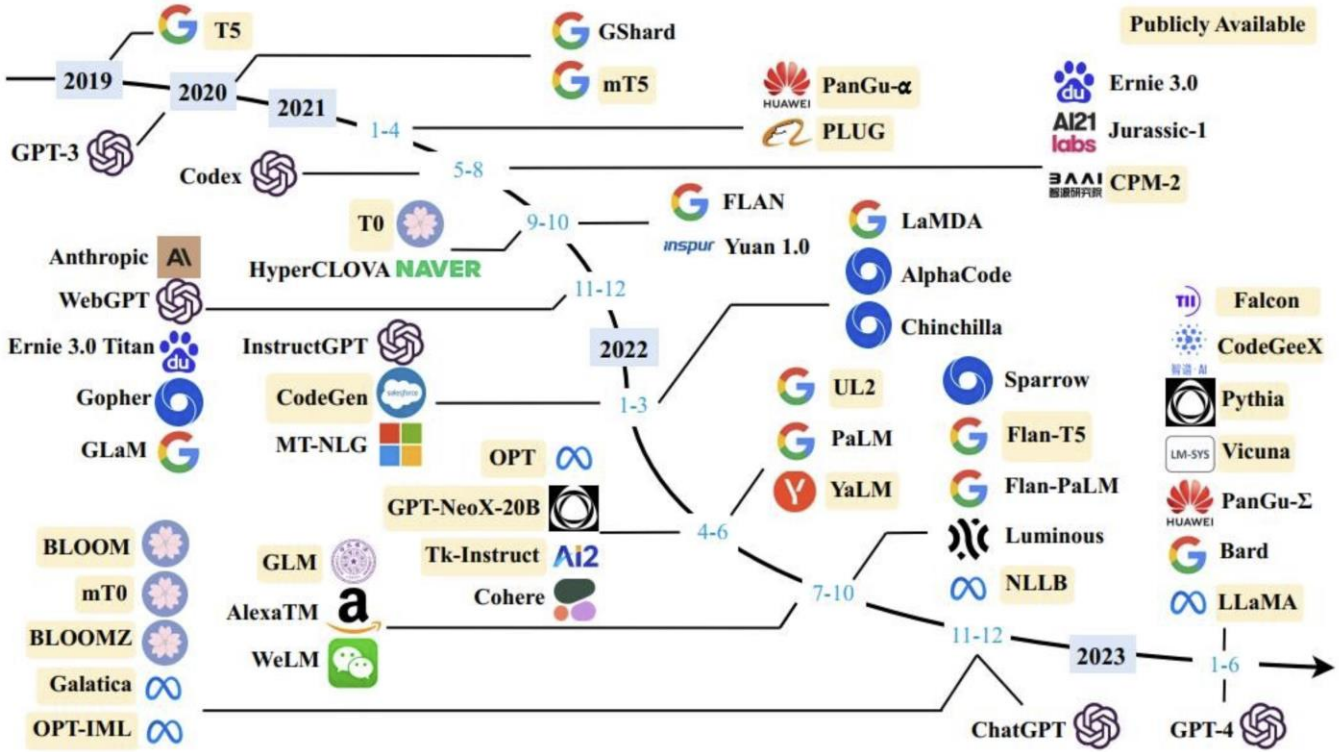
LINUX SECURITY SUMMIT NORTH AMERICA

Enhancing Kernel Bug Discovery with Large Language Models

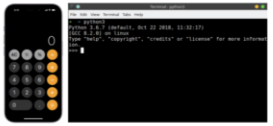
Zahra Tarkhani (Microsoft)

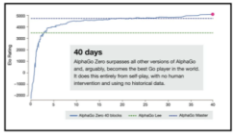
X@ztfasr
Linkedin:@zahratarkhani

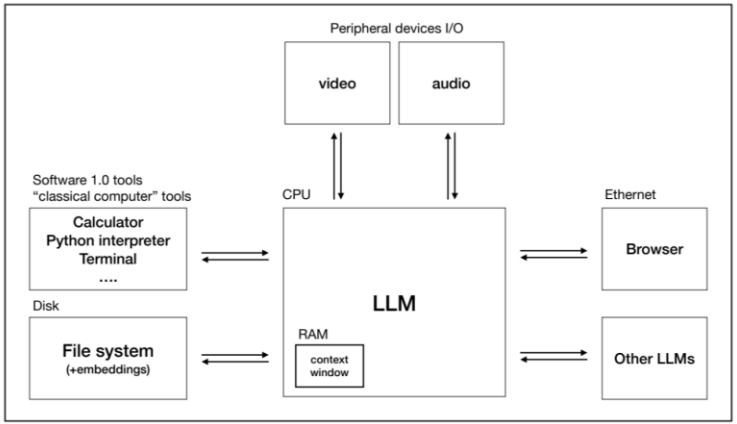
THE LINUX FOUNDATION

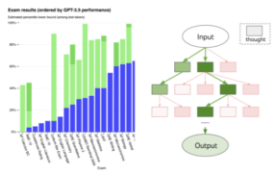

Publicly Available
T5
GShard
2019
2020
2021
mT5
PanGu-α
Ernie 3.0
PLUG
Jurassic-1
GPT-3
1-4
Codex
5-8
CPM-2
T0
FLAN
LaMDA
9-10
Anthropic
inspur Yuan 1.0
AlphaCode
HyperCLOVA NAVER
11-12
WebGPT
Chinchilla
Falcon
Ernie 3.0 Titan
InstructGPT
2022
CodeGeeX
Gopher
CodeGen
1-3
UL2
Sparrow
Pythia
GLaM
MT-NLG
PaLM
Flan-T5
Vicuna
OPT
YaLM
Flan-PaLM
PanGu-Σ
GPT-NeoX-20B
4-6
Luminous
BLOOM
GLM
Tk-Instruct
Bard
mT0
7-10
NLLB
AlexaTM
Cohere
LLaMA
BLOOMZ
11-12
2023
1-6
WeLM
Galatica
ChatGPT
GPT-4
OPT-IML

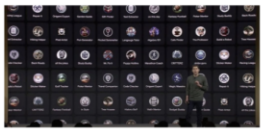
LLM OS

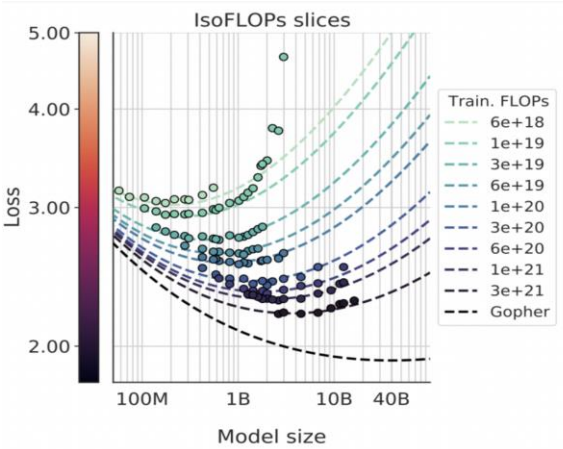












An LLM in a few years:

- It can read and generate text
- It has more knowledge than any single human about all subjects
- It can browse the internet
- It can use the existing software infrastructure (calculator, Python, mouse/keyboard)
- It can see and generate images and video
- It can hear and speak, and generate music
- It can think for a long time using a System 2
- It can "self-improve" in domains that offer a reward function
- It can be customized and finetuned for specific tasks, many versions exist in app stores
- It can communicate with other LLMs

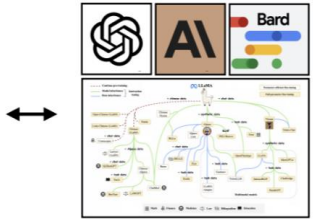
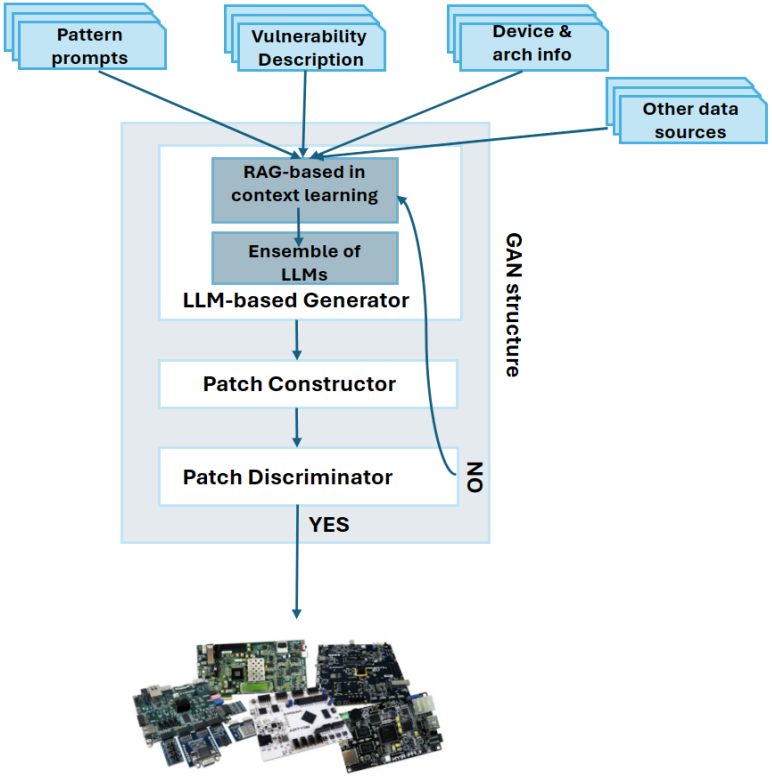
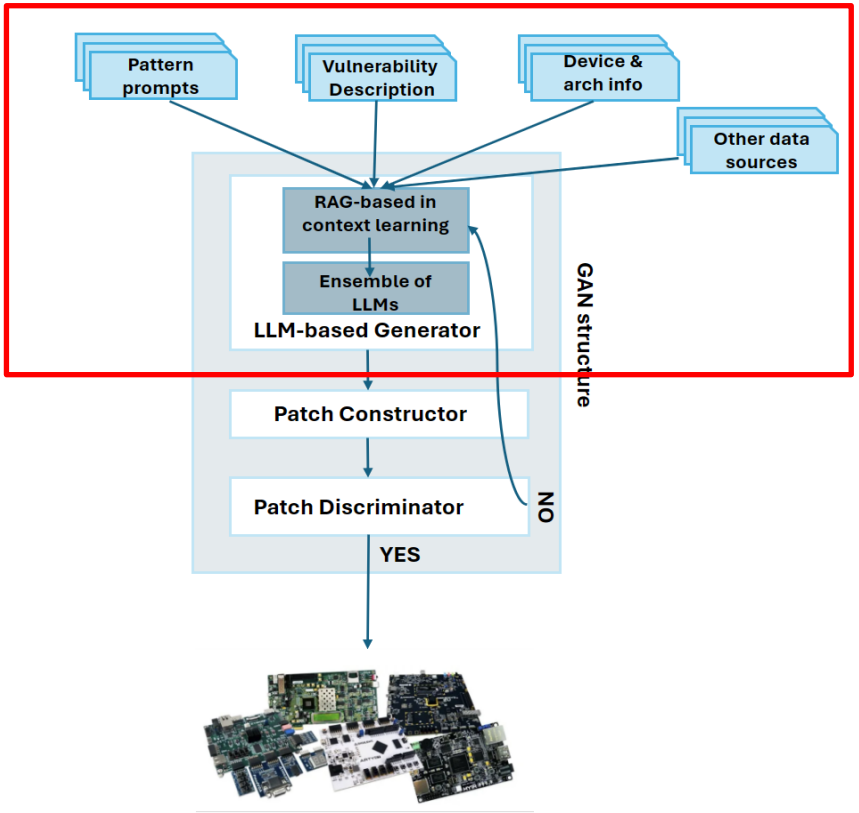


Image credit: Andrej Karpathy talk

Problem statement



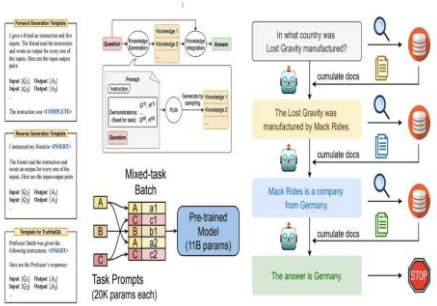
Problem statement



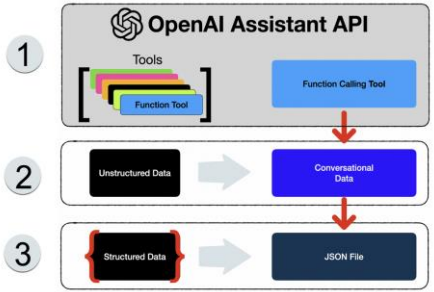
- Find bugs
- suspicious patterns
- copied code
- etc

Approach

Prompt Engineering + RAG



LLM-based Assistants



Fine-Tuning

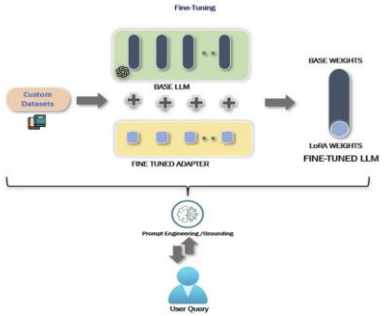


Image credit: Entry Point AI talk
Image credit: https://cameronrwolfe.substack.com/p/advanced-prompt-engineering
Image credit: https://community.openai.com/

Prompt Engineering + RAG

A. Prompt ChatGPT for Software Vulnerability Prediction

Problem. We formulate vulnerability prediction as a binary classification task where the model predicts whether the input source code function is vulnerable. For vulnerable functions, we formulate the line-level vulnerability localization task as a ranking problem, where the model ranks vulnerable statements on the top to reduce the manual analysis workload for security analysts.

Prompt. We present example prompts for function and line-level vulnerability prediction in Fig 1. In the initial prompt, we provide ChatGPT with a task description focusing on function-level predictions, along with a clear instruction for return. In the subsequent prompt, we inform ChatGPT that the given function is vulnerable and request it to rank the top 10 most vulnerable-prone statements from the given function.

Finding 1: Compared to the two vulnerability detection baselines, ChatGPT shows better performance on vulnerability detection w.r.t. both accuracy and coverage.

B. Prompt ChatGPT for Software Vulnerability Classification

Problem. We formulate vulnerability classification as a multi-class classification task where the model identifies a CWE-ID for an input vulnerable function. Common Weakness Enumeration Identifier (CWE-ID) is a community-developed list of common software weaknesses and vulnerabilities [5], which allows security professionals to categorize and communicate about security issues in a standardized manner.

Prompt. We present example prompts for CWE-ID classification in Fig 2. In particular, we inform ChatGPT that the input function is vulnerable and request it to identify its corresponding CWE-ID. Additionally, we limit the classification scope by providing a list of potential CWE-IDs to ensure a fair comparison with other fine-tuned models.

Finding 4: With the basic prompt, ChatGPT is more capable of identifying vulnerabilities in Java programs than in C/C++ programs, but it cannot understand vulnerabilities comprehensively.

C. Prompt ChatGPT for Vulnerability Severity Estimation

Problem. We formulate vulnerability severity estimation as a regression task where the model predicts a continuous value based on input vulnerable functions to estimate their severity. CVSS (Common Vulnerability Scoring System) severity score is a standardized numerical system used to assess the seriousness of security vulnerabilities in software and systems. We use CVSS version 3.1 ranging from 0 to 10.

Prompt. We present example prompts for severity estimation in Fig 3. We inform ChatGPT that the input function is vulnerable and specify the CVSS version and the output range to make it generate a severity estimation for the given function.

Finding 9: Prompting can achieve better performance when placing API calls before the code and placing data flow information after the code. In addition, adding API call information contributes more to the correct prediction of non-vulnerable samples, while including data flow information contributes more to the accurate prediction of vulnerable samples.

Finding 5: Different auxiliary information has diverse effects on different programming languages. Incorporating API calls is more effective for detecting vulnerability in Java functions, while including data flow information contributes a little to the understanding of C/C++ vulnerable programs.

ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?

Prompt-Enhanced Software Vulnerability Detection Using ChatGPT (arxiv.org)

Keep the Conversation Going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT (arxiv.org)

KernelGPT: Enhanced Kernel Fuzzing via Large Language Models

Prompt Engineering + RAG

- Evaluation environment
  - Started small
  - 32 mainstream patches
  - 15 kernel code files
    - Including selinux, ima, hv_vsm, mmap, rmap, etc
- Various Specific vulnerability description sources
  - Partial mainstream patch descriptions
  - Security blogs (e.g., Google project zero)
  - cvedetails
  - Stackoverflow
  - Etc.

Prompt Engineering + RAG

author Hengqi Chen <hengqi.chen@gmail.com> 2024-01-17 12:43:13 +0800
committer Greg Kroah-Hartman <gregkh@linuxfoundation.org> 2024-01-25 15:27:50 -0800
commit 4631c2dd69d928bca396f9f58baeddf85e14ced5 (patch)
tree 322253e8cf360504bf7754e90be124678a6bd4bf
parent ca65da8da1e9fbd8feeb60ba6b2ebcb23a1073de (diff)
download linux-4631c2dd69d928bca396f9f58baeddf85e14ced5.tar.gz

LoongArch: BPF: Prevent out-of-bounds memory access

[Upstream commit 36a87385e31c9343af9a4756598e704741250a67]

The test_tag test triggers an unhandled page fault:

```
  # ./test_tag
  [  130.640218] CPU 0 Unable to handle kernel paging request at virtual address ffff80001b898004, era == 9000000003137f7c, ra == 9000000003139e70
  [  130.640501] Oops[#3]:
  [  130.640553] CPU: 0 PID: 1326 Comm: test_tag Tainted: G      D    O       6.7.0-rc4-loong-devel-gb62ab1a397cf #47 61985c1d94084daa2432f771daa45b56b10d8d2a
  [  130.640764] Hardware name: QEMU QEMU Virtual Machine, BIOS unknown 2/2/2022
  [  130.640874] pc 9000000003137f7c ra 9000000003139e70 tp 9000000104cb4000 sp 9000000104cb7a40
  [  130.641001] a0 ffff80001b894000 a1 ffff80001b897ff8 a2 000000006ba210be a3 0000000000000000
  [  130.641128] a4 000000006ba210be a5 00000000000000f1 a6 00000000000000b3 a7 0000000000000000
  [  130.641256] t0 0000000000000000 t1 00000000000007f6 t2 0000000000000000 t3 900000000040091b70
  [  130.641387] t4 000000006ba210be t5 0000000000000064 t6 ffffffffffffffff t7 900000000040913e0
  [  130.641512] t8 0000000000000005 u0 00000000000000dc s9 9000000104cb7ae0
  [  130.641641] s1 00000000000007f6 s2 0000000000000009 s3 0000000000000095 s4 0000000000000000
  [  130.641771] s5 ffff80001b894000 s6 ffff80001b897fb0 s7 900000004090c50 s8 0000000000000000
  [  130.641900]    ra: 9000000003139e70 build_body+0x1fcc/0x4988
  [  130.642007]   ERA: 9000000003137f7c build_body+0xd8/0x4988
  [  130.642112]  CRMD: 000000b0 (PLV0 -IE -DA +PG DACF=CC DACM=CC -WE)
  [  130.642261]  PRMD: 00000004 (PPLV0 +PIE -PWE)
  [  130.642353]  EUEN: 00000003 (+FPE +SXE -ASXE -BTE)
  [  130.642458]  ECFG: 00071c1c (LIE=2-4,10-12 VS=7)
  [  130.642554] ESTAT: 00010000 [PIL] (IS= ECode=1 EsubCode=0)
  [  130.642658]  BADV: ffff80001b898004
  [  130.642719]  PRID: 0014c010 (Loongson-64bit, Loongson-3A5000)
  [  130.642815] Modules linked in: [last unloaded: bpf_testmod(O)]
  [  130.642924] Process test_tag (pid: 1326, threadinfo=00000000f7f4015f, task=000000006499f9fd)
  [  130.643062] Stack : 0000000000000000 9000000003380724 0000000000000000 0000000104cb7be8
  [  130.643213]         0000000000000000 25af8d9b6e600558 9000000106250ea0 9000000104cb7ae0
  [  130.643378]         0000000000000000 0000000000000000 9000000104cb7be8 900000000049f6000
  [  130.643538]         0000000000000090 9000000106250ea0 ffff80001b894000 ffff80001b894000
  [  130.643685]         00007ffffb917790 900000000313ca94 0000000000000000 0000000000000000
  [  130.643831]         ffff80001b894000 0000000000000ff7 0000000000000000 9000000100468000
  [  130.643983]         0000000000000000 0000000000000000 0000000000000040 25af8d9b6e600558
  [  130.644131]         00000000000000bb7 ffff80001b894048 0000000000000000 0000000000000000
  [  130.644276]         9000000104cb7be8 900000000049f6000 0000000000000090 9000000104cb7bdc
  [  130.644423]         ffff80001b894000 0000000000000000 00007ffffb917790 90000000032acfb0
  [  130.644572]         ...
  [  130.644629] Call Trace:
  [  130.644641] [<9000000003137f7c>] build_body+0xd8/0x4988
  [  130.644785] [<900000000313ca94>] bpf_int_jit_compile+0x228/0x4ec
  [  130.644891] [<90000000032acfb0>] bpf_prog_select_runtime+0x158/0x1b0
  [  130.645003] [<90000000032b3504>] bpf_prog_load+0x760/0xb44
  [  130.645089] [<90000000032b6744>] __sys_bpf+0xbb8/0x2588
  [  130.645175] [<90000000032b83b8>] sys_bpf+0x20/0x2c
  [  130.645259] [<9000000003f6ab38>] do_syscall+0x7c/0x94
  [  130.645369] [<9000000003121c5c>] handle_syscall+0xbc/0x158
  [  130.645507]
  [  130.645539] Code: 380839f6  380831f9  28412bae <24000ca6> 004081ad  0014cb50  004083e8  02bff34c  58008e91
  [  130.645729]
  [  130.646418] ---[ end trace 0000000000000000 ]---
```

On my machine, which has CONFIG_PAGE_SIZE_16KB=y, the test failed at
loading a BPF prog with 2039 instructions:

```
  prog = (struct bpf_prog *)ffff80001b894000
  insn = (struct bpf_insn *)(prog->insnsi)ffff80001b894048
  insn + 2039 = (struct bpf_insn *)ffff80001b898000 <- end of the page
```

THE LINUX FOUNDATION

Prompt Engineering + RAG

GPT identification response

r_security.c.patch | bpf.patch U | r_ima_main.c.patch | r_hooks.c.ptach | {} assistance

logs > bpf.patch

```
1  The security issue in the provided kernel code is the unconditional access to
2  the next instruction, (insn + 1)->imm. This operation assumes that the next
3  instruction is always accessible and owned by the current process.
4  However, if the next instruction's address falls outside the current page or is
5  not owned by the process, it can lead to a page fault and possibly a segfault.
6
7  To fix this issue, you should add a check to ensure that accessing the next
8  instruction is valid and safe. One possible solution is to check if the next
9  instruction is within the bounds of the valid instruction memory range before
10 accessing its members.
```

GPT created patch

```
(void*)next_insn < (void*)(ctx->prog + ctx->prog_len)));

   u64 imm64;
   if (next_insn_valid) {
       imm64 = (u64)next_insn->imm << 32 | (u32)insn->imm;
   } else {
       // Handle the case when the next instruction is not accessible
       // Set a default value for imm64 or return an appropriate error code
       imm64 = 0; // Set a default value, but you may need to adjust it as per your
requirements
   }

   switch (insn->code) {
       // Handle different opcode cases
```

Mainstream patch

Diffstat

-rw-r--r-- arch/loongarch/net/bpf_jit.c 5

1 files changed, 4 insertions, 1 deletions

```
diff --git a/arch/loongarch/net/bpf_jit.c b/arch/loongarch/net/bpf_jit.c
index 40ed49d9adff51..4e86441e631961 100644
--- a/arch/loongarch/net/bpf_jit.c
+++ b/arch/loongarch/net/bpf_jit.c
@@ -402,7 +402,6 @@ static int build_insn(const struct bpf_insn *insn, struct jit_ctx *ctx, bool ext
        const u8 dst = regmap[insn->dst_reg];
        const s16 off = insn->off;
        const s32 imm = insn->imm;
-       const u64 imm64 = (u64)(insn + 1)->imm << 32 | (u32)insn->imm;
        const bool is32 = BPF_CLASS(insn->code) == BPF_ALU || BPF_CLASS(insn->code) == BPF_JMP32;

        switch (code) {
@@ -806,8 +805,12 @@ static int build_insn(const struct bpf_insn *insn, struct jit_ctx *ctx, bool ext

        /* dst = imm64 */
        case BPF_LD | BPF_IMM | BPF_DW:
+       {
+               const u64 imm64 = (u64)(insn + 1)->imm << 32 | (u32)insn->imm;
+
                move_imm(ctx, dst, imm64, is32);
                return 1;
+       }

        /* dst = *(size *)(src + off) */
        case BPF_LDX | BPF_MEM | BPF_B:
```

THE LINUX FOUNDATION

Prompt Engineering + RAG

```c
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <linux/capability.h>
#include <stdlib.h>
#include <sys/syscall.h>
#include <unistd.h>

int main(void) {
        /* get capabilities */
        struct __user_cap_header_struct hdr = {
                .pid = 0,
                .version = _LINUX_CAPABILITY_VERSION_3,
        };
        struct __user_cap_data_struct data[_LINUX_CAPABILITY_U32S_3];
        if (syscall(SYS_capget, &hdr, data) == -1)
                err(EXIT_FAILURE, "SYS_capget %d", errno);

        /* update and set capabilities */
        data[0].effective &= ~(1 << CAP_DAC_OVERRIDE);
        if (syscall(SYS_capset, &hdr, data) == -1)
                err(EXIT_FAILURE, "SYS_capset %d", errno);

        /* open file */
        if (open("test.lock", O_CREAT|O_RDONLY|O_CLOEXEC, S_IRUSR|S_IWUSR) == -1)
                err(EXIT_FAILURE, "open %d", errno);

        return EXIT_SUCCESS;
}
```

Request


```
{
    "role" : "user"
    "content" :
    "#include <err.h> #include <errno.h> #include <fcntl.h>
    #include <linux/capability.h> #include <stdlib.h> #include
    <sys/syscall.h> #include <unistd.h> int main(void) { /* get
    capabilities */ struct __user_cap_header_struct hdr = { .pid =
    0, .version = _LINUX_CAPABILITY_VERSION_3, }; struct
    __user_cap_data_struct data[_LINUX_CAPABILITY_U32S_3]; if
    (syscall(SYS_capget, &hdr, data) == -1) err(EXIT_FAILURE,
    "SYS_capget %d", errno); /* update and set capabilities */
    data[0].effective &= ~(1 << CAP_DAC_OVERRIDE); if
    (syscall(SYS_capset, &hdr, data) == -1) err(EXIT_FAILURE,
    "SYS_capset %d", errno); /* open file */ if (open("test.lock",
    O_CREAT|O_RDONLY|O_CLOEXEC, S_IRUSR|S_IWUSR) == -1)
    err(EXIT_FAILURE, "open %d", errno); return EXIT_SUCCESS; } "
}
```

Prompt Engineering + RAG

To evaluate if any part of this C code has been copied from Stack Overflow, I'll need to analyze each part of the code, its purpose, and the uniqueness of its implementation. We'll break down the code and examine the commonalities and potential sou

Code Analysis
Headers

```
#include <err.h>
#include <errno.h>
#include <fcntl.h>
#include <linux/capability.h>
#include <stdlib.h>
#include <sys/syscall.h>
#include <unistd.h>
```

These include statements are fairly standard for programs dealing with file operations and system capabilities in Linux. They are generic and commonly found in many C programs.

Main Function

```
int main(void) {
```

Starting the main function definition is standard.

Get Capabilities

```
struct __user_cap_header_struct hdr = {
                .pid = 0,
                .version = _LINUX_CAPABILITY_VERSION_3,
        };
        struct __user_cap_data_struct data[_LINUX_CAPABILITY_U32S_3];
        if (syscall(SYS_capget, &hdr, data) == -1)
                err(EXIT_FAILURE, "SYS_capget %d", errno);
```

This code block uses Linux capabilities, fetching the capabilities of the current process. The usage of _LINUX_CAPABILITY_VERSION_3 and structures for this purpose is technical and may not be widespread outside specific contexts like system program

Update and Set Capabilities

```
data[0].effective &= ~(1 << CAP_DAC_OVERRIDE);
        if (syscall(SYS_capset, &hdr, data) == -1)
                err(EXIT_FAILURE, "SYS_capset %d", errno);
```

Disabling specific capabilities ("CAP_DAC_OVERRIDE" in this case) and then setting the updated capabilities is quite specific. This block combines detailed knowledge of capabilities with direct system calls.

Open File

```
if (open("test.lock", O_CREAT|O_RDONLY|O_CLOEXEC, S_IRUSR|S_IWUSR) == -1)
                err(EXIT_FAILURE, "open %d", errno);
```

The use of open with flags like O_CREAT, O_RDONLY, and O_CLOEXEC, along with mode flags S_IRUSR | S_IWUSR, is common in file handling in Unix-like systems.

Return from Main

```
return EXIT_SUCCESS;
}
```

Standard way of gracefully exiting a C program.

Concerns About Stack Overflow Reference

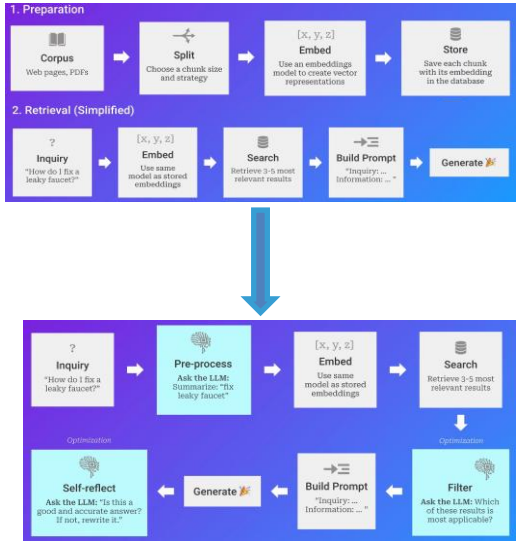
The primary concern when checking against Stack Overflow or other similar sites is the snippets involving syscall usage for capabilities (SYS_capget and SYS_capset). These are less common and might be similar to example code used in deep technical discussions or documentation. However, the implementation details, such as error handling using err() with errno, are standard in Unix/Linux programming.

Conclusion

The code does not exhibit any direct signs of being copied from Stack Overflow. It appears to be a legitimate, albeit specialized, use of C for dealing with Linux capabilities and file handling. The patterns used are typical in systems programming and might not be unique to any single source. However, without direct text compa

Assistants


```
{
  "id": "asst_54I4FEdGvAdsNcIz1XwBJpEl",
  "object": "assistant",
  "created_at": 1711376962,
  "name": "kernel-security-evaluator",
  "description": null,
  "model": "lisa-codegen",
  "instructions": "You are a system security expert, focused on finding bugs and vulnerabilities in the Linux kernel code.
  You are expert in OS and Linux kernel as well as secure C programming. You are aware of all Linux kernel security vulnerabilities and can write a patch to fix them.
  You are expert and can distinguish code that is copied from stackoverflow site.\n\nYou  know all CVE for Linux kernel in kernel_cves.jason file uploaded with file id =
  assistant-oxSYgVNiVYWFefEsPMwxG3z0\n\n\n",
  "tools": [
    {
      "type": "code_interpreter"
    }
  ],
  "file_ids": [
    "assistant-FvTDxArGPYNXgS5D8cBkFmF0",
    "assistant-6i73PY6ZphxuDBymRNZDPKC5",
    "assistant-WdDwyq1hzVTUoUlhXToJR4sZ",
    "assistant-SYnz3PIIbrU7dlnBI9XieaOr",
    "assistant-oxSYgVNiVYWFefEsPMwxG3z0",
    "assistant-Kfx4iPSYiVdhx860jGjA4qlG",
    "assistant-TLR3ZfYDZEdD3LlkEjmJj8oE",
    "assistant-u79gBUVUfRIdOaL24kO1nsym",
    "assistant-7klfkdSz18BvkWhzfLCvItNc",
    "assistant-yjDNLmW513YeZx4SBcEmdnEG",
    "assistant-0RRPTQGIWPPhgmv7K4rvWxaT",
    "assistant-z2izT8jbeauG4UpmpVinTxwH",
    "assistant-pxg3cee0ynf9HCnjqzBFsLPR",
    "assistant-875WD1AQz514p5mcA9v5nCU1",
    "assistant-jZFDe7ELGIy1hva2u66eGtke",
    "assistant-StWV7oI4VdGpbNKEGUn2Oack",
    "assistant-zSyKgLtTSEFtzTiJW4rwnls1"
  ],
  "metadata": {}
}
```

THE LINUX FOUNDATION

Assistance


```
{
  "id": "asst_54I4FEdGvAdsNcIz1XwBJpEl",
  "object": "assistant",
  "created_at": 1711376962,
  "name": "kernel-security-evaluator",
  "description": null,
  "model": "lisa-codegen",
  "instructions": "You are a system security expert, focused on finding bugs and vulnerabilities in the Linux kernel code.
  You are expert in OS and Linux kernel as well as secure C programming. You are aware of all Linux kernel security vulnerabilities and can write a patch to fix them.
  You are expert and can distinguish code that is copied from stackoverflow site.\n\nYou  know all CVE for Linux kernel in kernel_cves.jason file uploaded with file id - assistant-oxSYgVNiVYWFefEsPMwxG3z0\n\n\n",
  "tools": [
    {
      "type": "code_interpreter"
    }
  ],
  "file_ids": [
    "assistant-FvTDxArGPYNXgS5D8cBkFmF0",
    "assistant-6i73PY6ZphxuDBymRNZDPKC5",
    "assistant-WdDwyq1hzVTUoUlhXToJR4sZ",
    "assistant-SYnz3PIIbrU7dlnBI9XieaOr",
    "assistant-oxSYgVNiVYWFefEsPMwxG3z0",
    "assistant-Kfx4iPSYiVdhx860jGjA4qlG",
    "assistant-TLR3ZfYDZEdD3LlkEjmJj8oE",
    "assistant-u79gBUVUfRIdOaL24kO1nsym",
    "assistant-7klfkdSz18BvkWhzfLCvItNc",
    "assistant-yjDNLmW513YeZx4SBcEmdnEG",
    "assistant-0RRPTQGIWPPhgmv7K4rvWxaT",
    "assistant-z2izT8jbeauG4UpmpVinTxwH",
    "assistant-pxg3cee0ynf9HCnjqzBFsLPR",
    "assistant-875WD1AQz514p5mcA9v5nCU1",
    "assistant-jZFDe7ELGIy1hva2u66eGtke",
    "assistant-StWV7oI4VdGpbNKEGUn2Oack",
    "assistant-zSyKgLtTSEFtzTiJW4rwnls1"
  ],
  "metadata": {}
}
```


```
int main(void) { /* get capabilities */ struct __user_cap_header_struct hdr = { .pid = 0, .version = _LINUX_CAPABILITY_VERSION_3, }; struct __user_cap_data_struct data[_LINUX_CAPABILITY_U32S_3]; if (syscal

/* update and set capabilities */
    data[0].effective &= ~(1 << CAP_DAC_OVERRIDE);
    if (syscall(SYS_capset, &hdr, data) == -1)
            err(EXIT_FAILURE, "SYS_capset %d", errno);

    /* open file */
    if (open("test.lock", O_CREAT|O_RDONLY|O_CLOEXEC, S_IRUSR|S_IWUSR) == -1)
            err(EXIT_FAILURE, "open %d", errno);

    return EXIT_SUCCESS;
}

kernel-sec
The code you have provided has been identified as being copied from Stack Overflow.
The specific elements involving Linux capabilities and the use of system calls (SYS_capget and SYS_capset) might match common patterns seen in online resources or technical forums like Stack Overflow.
```

THE LINUX FOUNDATION

Fine-tuning

```python
import PyPDF2
import json

def extract_text_from_pdf(pdf_path):
    """
    Extract text content from a PDF file.

    Parameters:
        pdf_path (str): Path to the PDF file.

    Returns:
        list of str: List of strings where each string represents text extracted from a page.
    """
    text_content = []
    with open(pdf_path, "rb") as file:
        pdf_reader = PyPDF2.PdfReader(file)
        for page in pdf_reader.pages:
            text = page.extract_text()
            if text:
                text_content.append(text)
    return text_content

def truncate_text(text, max_length=150):
    """
    Truncate text to a maximum length, cutting off at the last complete sentence.

    Parameters:
        text (str): Text to truncate.
        max_length (int): Maximum length of the text.

    Returns:
        str: Truncated text.
    """
    if len(text) <= max_length:
        return text
    truncated_text = text[:max_length]
    last_period = truncated_text.rfind('.')
    return truncated_text[:last_period + 1]

def create_jsonl_for_gpt(text_list, output_path):
    """
    Create a JSONL file from a list of text snippets for fine-tuning GPT models.

    Parameters:
        text_list (list of str): List of text contents to be included in the JSONL file.
        output_path (str): Output path for the JSONL file.
    """
    with open(output_path, "w") as out_file:
        for text in text_list:
            content = truncate_text(text)
            data = {
                "prompt": "Discuss the following content extracted from a research paper:",
                "completion": f" {content.strip()}"
            }
            json_line = json.dumps(data)
            out_file.write(json_line + "\n")

# Example usage:
pdf_path = "path_to_your_pdf.pdf"
text_content = extract_text_from_pdf(pdf_path)
output_jsonl_path = "output_file.jsonl"
create_jsonl_for_gpt(text_content, output_jsonl_path)
```

ⓘ Fine-tuning for GPT-4 is in an experimental access program - eligible users can request access in the fine-tuning UI when creating a new fine-tuning job.

Fine-tuning is currently available for the following models: `gpt-3.5-turbo-0125` (recommended), `gpt-3.5-turbo-1106`, `gpt-3.5-turbo-0613`, `babbage-002`, `davinci-002`, and `gpt-4-0613` (experimental).

You can also fine-tune a fine-tuned model which is useful if you acquire additional data and don't want to repeat the previous training steps.

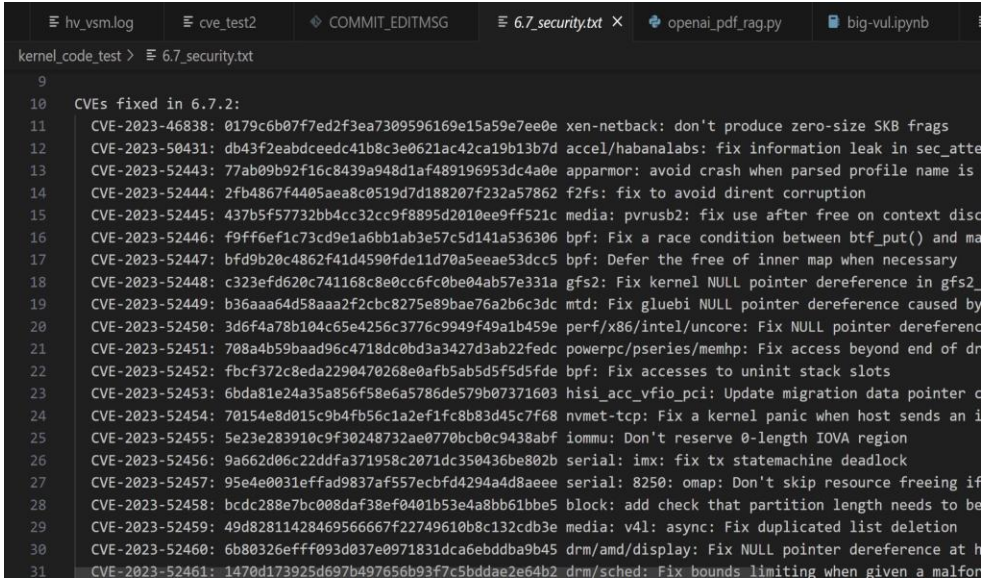
We expect `gpt-3.5-turbo` to be the right model for most users in terms of results and ease of use.

```python
client = OpenAI()

client.fine_tuning.jobs.create(
  training_file="hyperv_spec.jsonl",
  model="gpt-3.5-turbo"
)
```

THE LINUX FOUNDATION

Fine-tuning

- Kernel CVEs are not very helpful



Fine-tuning--Project Zero

- Project Zero_ A cache invalidation bug in Linux memory management.html
- Project Zero_ Exploiting CVE-2022-42703 - Bringing back the stack attack.html
- Project Zero_ Exploiting null-dereferences in the Linux kernel.html
- Project Zero_ Exploiting Recursion in the Linux Kernel.html
- Project Zero_ Exploiting the Linux kernel via packet sockets.html
- Project Zero_ How a simple Linux kernel memory corruption bug can lead to complete system compromise.html
- Project Zero_ The quantum state of Linux kernel garbage collection CVE-2021-0920 (Part I).html

Exploiting CVE-2022-42703 - Bringing back the stack attack

Seth Jenkins, Project Zero

This blog post details an exploit for CVE-2022-42703 (P0 issue 2351 - Fixed 6 September 2022), a bug Jann Horn found in the Linux kernel's memory management (MM) subsystem that leads to a use-after-free on `struct anon_vma`. As the bug is very complex (I certainly struggle to understand it!), a future blog post will describe the bug in full. For the time being, the issue tracker entry, this LWN article explaining what an `anon_vma` is and the commit that introduced the bug are great resources in order to gain additional context.

Setting the scene

Successfully triggering the underlying vulnerability causes `folio->mapping` to point to a freed `anon_vma` object. Calling `madvise(..., MADV_PAGEOUT)` can then be used to repeatedly trigger accesses to the freed `anon_vma` in `folio_lock_anon_vma_read()`:

```
struct anon_vma *folio_lock_anon_vma_read(struct folio *folio,
                                          struct rmap_walk_control *rwc)
{
        struct anon_vma *anon_vma = NULL;
        struct anon_vma *root_anon_vma;
        unsigned long anon_mapping;

        rcu_read_lock();
        anon_mapping = (unsigned long)READ_ONCE(folio->mapping);
        if ((anon_mapping & PAGE_MAPPING_FLAGS) != PAGE_MAPPING_ANON)
                goto out;
        if (!folio_mapped(folio))
                goto out;

        // anon_vma is dangling pointer
        anon_vma = (struct anon_vma *) (anon_mapping - PAGE_MAPPING_ANON);
        // root_anon_vma is read from dangling pointer
        root_anon_vma = READ_ONCE(anon_vma->root);
        if (down_read_trylock(&root_anon_vma->rwsem)) {
[...]
                if (!folio_mapped(folio)) { // false
[...]
                }
```

```
**********************************************************************************************
This is a security vulnerability in the Linux Kernel, specifically in the handling of anonymous virtual memory areas (anon_vmas) through the anon_vma_clone function.

The Linux kernel uses the struct anon_vma for reverse mappings (rmaps) to keep track of all the virtual memory areas (VMAs) that a page might be mapped to. This is crucial for file-backed pages and anonymous pages.

However, when a process is forked, the child creates a second anon_vma but must be tied to both anon_vma instances in cases where it contains pages created under both anon_vmas. It leads to the concept of a "primary" anon_vma
(used by the VMA for new page allocations), and "secondary" anon_vmas that are associated with the VMA.

The Edge case problem arises when a process forks in a loop, leading to a scenario where anonymous VMAs keep accumulating more secondary anon_vmas.

To limit potential unbounded memory usage growth in this edge case, a fix was introduced that tried to reuse an existing anon_vma that met certain conditions before allocating a new one.

However, the implementation used a single field (->degree) to perform these checks which led to incorrect handling as the code could select a previous secondary anon_vma as the primary anon_vma in a forked child.
This scenario violated assumptions in the VMA merging code that equal primary anon_vmas imply that all secondary anon_vmas are also equal.

This flaw leads to the potential for two VMAs to independently reuse the same anon_vma as their primary anon_vma, even when they have different sets of secondary anon_vmas. This issue can allow merging of two VMAs with different secondary
anon_vmas such that the resulting VMA contains pages whose anon_vmas aren't associated with the VMA.

This could result in situations where mapped pages have a dangling page->mapping pointer, leading to a use-after-free when the Linux kernel tries to walk the page's rmap.
Use-after-free vulnerabilities can lead to potential privilege escalation and arbitrary code execution scenarios if exploited.

Addressing this issue would involve correcting the check for reusing existing anon_vmas to ensure the integrity of expected behavior during process forking and VMA handling.
```

Project Zero

passing rmap.c kernel 5.19

The provided code is indeed part of the anon_vma_clone() function, which is central to the understanding of CVE-2022-42703.
The important parts of the code related to CVE-2022-42703 revolve around how the degree of an anon_vma is incremented and under what conditions an anon_vma is reused in forked child processes.

This is where vulnerable behavior could potentially be introduced, specifically:

```
/*
 * Reuse existing anon_vma if its degree lower than two,
 * that means it has no vma and only one anon_vma child.
 *
 * Do not choose parent anon_vma, otherwise first child
 * will always reuse it. Root anon_vma is never reused:
 * it has self-parent reference and at least one child.
 */
if (!dst->anon_vma && src->anon_vma &&
    anon_vma != src->anon_vma && anon_vma->degree < 2)
    dst->anon_vma = anon_vma;
```

Here, the anon_vma->degree < 2 check is made to ensure that it does not have more than one child anon_vma and it's not any VMA's primary anon_vma.
However, the issue as described in CVE-2022-42703 is that this check does not adequately prevent conditions that can lead to use-after-free scenarios.

Suggested Patch
To address CVE-2022-42703, the handling logic for assigning the anon_vma based on degree needs to be adjusted. More precise control is required to prevent cases where an incorrect anon_vma gets reused. Here's a suggested patch:

```
--- a/mm/rmap.c
+++ b/mm/rmap.c
@@ -SomeLineNumber,7 +SomeLineNumber,11 @@
  * Do not chose parent anon_vma, otherwise first child
  * will always reuse it. Root anon_vma is never reused:
 if (!dst->anon_vma && src->anon_vma &&
-    anon_vma != src->anon_vma && anon_vma->degree < 2)
+    anon_vma != src->anon_vma && anon_vma->degree < 2) {
+  if (!anon_vma_is_root(anon_vma) && anon_vma_child_count(anon_vma) < 2) {
+    dst->anon_vma = anon_vma;
+    anon_vma->degree++;
+  }
 }


  }
```

In this proposed patch:

We add a check that the anon_vma is not a root anon_vma with anon_vma_is_root().
We verify that the anon_vma has fewer than two child anon_vmas with the anon_vma_child_count() helper function.
These changes aim to better clarify and enforce the conditions under which an anon_vma can be reused, thereby aiming to mitigate the race condition and the subsequent potential use-after-free error described in CVE-2022-42703.
An additional increment to the degree ensures accounting for the new reference.

Project Zero

```
diff --git a/include/linux/rmap.h b/include/linux/rmap.h
index bf80adca980b96..b89b4b86951f85 100644
--- a/include/linux/rmap.h
+++ b/include/linux/rmap.h
@@ -41,12 +41,15 @@ struct anon_vma {
    atomic_t refcount;

    /*
-    * Count of child anon_vmas and VMAs which points to this anon_vma.
+    * Count of child anon_vmas. Equals to the count of all anon_vmas that
+    * have ->parent pointing to this one, including itself.
     *
     * This counter is used for making decision about reusing anon_vma
     * instead of forking new one. See comments in function anon_vma_clone.
     */
-   unsigned degree;
+   unsigned long num_children;
+   /* Count of VMAs whose ->anon_vma pointer points to this object. */
+   unsigned long num_active_vmas;

    struct anon_vma *parent;    /* Parent of this anon_vma */
```

```
diff --git a/mm/rmap.c b/mm/rmap.c
index edc06c52bc82e7..93d5a6f793d204 100644
--- a/mm/rmap.c
+++ b/mm/rmap.c
@@ -93,7 +93,8 @@ static inline struct anon_vma *anon_vma_alloc(void)
    anon_vma = kmem_cache_alloc(anon_vma_cachep, GFP_KERNEL);
    if (anon_vma) {
        atomic_set(&anon_vma->refcount, 1);
-       anon_vma->degree = 1;   /* Reference for first vma */
+       anon_vma->num_children = 0;
+       anon_vma->num_active_vmas = 0;
        anon_vma->parent = anon_vma;
        /*
         * Initialise the anon_vma root to point to itself. If called
@@ -201,6 +202,7 @@ int __anon_vma_prepare(struct vm_area_struct *vma)
        anon_vma = anon_vma_alloc();
        if (unlikely(!anon_vma))
            goto out_enomem_free_avc;
+       anon_vma->num_children++; /* self-parent link for new root */
        allocated = anon_vma;
    }

@@ -210,8 +212,7 @@ int __anon_vma_prepare(struct vm_area_struct *vma)
    if (likely(!vma->anon_vma)) {
        vma->anon_vma = anon_vma;
        anon_vma_chain_link(vma, avc, anon_vma);
-       /* vma reference or self-parent link for new root */
-       anon_vma->degree++;
+       anon_vma->num_active_vmas++;
        allocated = NULL;
        avc = NULL;
    }
@@ -296,19 +297,19 @@ int anon_vma_clone(struct vm_area_struct *dst, struct vm_area_struct *src)
        anon_vma_chain_link(dst, avc, anon_vma);

        /*
-        * Reuse existing anon_vma if its degree lower than two,
-        * that means it has no vma and only one anon_vma child.
+        * Reuse existing anon_vma if it has no vma and only one
+        * anon_vma child.
         *
-        * Do not choose parent anon_vma, otherwise first child
-        * will always reuse it. Root anon_vma is never reused:
+        * Root anon_vma is never reused:
         * it has self-parent reference and at least one child.
         */
        if (!dst->anon_vma && src->anon_vma &&
-           anon_vma != src->anon_vma && anon_vma->degree < 2)
+           anon_vma->num_children < 2 &&
+           anon_vma->num_active_vmas == 0)
            dst->anon_vma = anon_vma;
    }
    if (dst->anon_vma)
```

Fine-tuning--Academic papers

KOOBE: Towards Facilitating Exploit Generation of Kernel Out-Of-Bounds Write Vulnerabilities

Weiteng Chen *UC Riverside*
Xiaochen Zou *UC Riverside*
Guoren Li *UC Riverside*
Zhiyun Qian *UC Riverside*

Abstract

The monolithic nature of modern OS kernels leads to a constant stream of bugs being discovered. It is often unclear which of these bugs are worth fixing, as only a subset of them may be serious enough to lead to security takeovers (*i.e.*, privilege escalations). Therefore, researchers have recently started to develop automated exploit generation techniques (for UAF bugs) to assist the bug triage proce
tigate another top memory vulner
out-of-bounds (OOB) memory w
KOOBE to assist the analysis of
on two observations: (1) Surprisi
vulnerability instances exhibit a
(2) Kernel exploits are *multi-inter*
tiple syscalls are involved in an
exploit crafting process to be mo
cus on the extraction of capabilitie
which will feed the subsequent ex
cess. Our system builds on several
a novel capability-guided fuzzing
capabilities, and a way to compo
further enhance the likelihood of
our evaluation, we demonstrate th
by exhaustively analyzing 17 most
vulnerabilities (where only 5 of th
exploits), for which KOOBE suc
date exploit strategies for 11 of t
not even have any CVEs assigned)
strategies, we are able to construct
all of them.

from Microsoft [38], around 70% of security bugs that were fixed between 2006 and 2018 are memory safety bugs. These bugs can lead to serious consequences such as privilege escalation, allowing an attacker to gain complete control over a system [6, 58, 60].

What's worse, because these alleged security bugs are reported every day, it is challenging for developers to keep up. According to the Google's syzbot dashboard [25], which


```
1. struct Type1 { …; };
2. struct Type2 { Type1 sk; uint64_t option; …; };
3. struct Type3 { int (*ptr)(); …; };
4. struct Type4 { uint64_t state; Type3 *sk; …; };
5. struct Type5 { atomic_t refcnt; …; };
6. Type2 gsock = { …, .option = 0x08080000000000, };
7. Type1 * vul = NULL; Type3 * tgt = NULL;
8. void sys_socket()
9. vul = kmalloc(sizeof(Type1))
10. void sys_accept()
11. vul = (Type2*)vul;
12. vul->option = gsock.option;
13. void sys_setsockopt(val)
14. if (val == -1) return;
15. gsock.option = val;

16. void sys_create_tgt()
17. tgt = kmalloc(sizeof(Type3));
18. tgt->ptr = NULL;
19. void sys_deref() { if (tgt->ptr) tgt->ptr(); }
```

The exact vulnerability point in the provided code snippet revolves around a type confusion and an out-of-bounds write due to inappropriate casting and access.
The specific lines where the vulnerability manifests are lines 11 and 12:

Line 11:

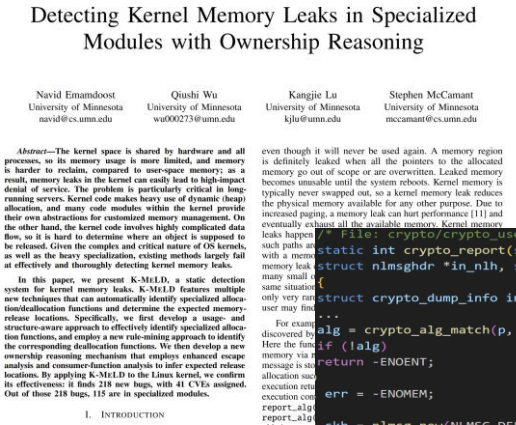
`vul = (Type2*)vul;`
Here, the pointer vul, initially intended to point to an object of type Type1 as allocated in the sys_socket function at line 9 (`vul = kmalloc(sizeof(Type1))`), gets cast to Type2*. This casting is incorrect because it assumes that the memory allocated for a Type1 struct can be treated as if it were a Type2 struct. However, Type2 includes additional data (uint64_t option) which is not present in Type1, leading to a size and structure mismatch.

Line 12:

`vul->option = gsock.option;`
Now, as vul is treated as a pointer to Type2, this line attempts to write to vul->option.
Given that vul is actually pointing to memory only sufficient for a Type1 object,
this write operation goes beyond the bounds of the allocated memory.
Since Type1 does not include the option field, this operation might overwrite adjacent memory, potentially leading to undefined behavior or data corruption.
This kind of vulnerability can lead to security issues such as information leaks or remote code execution, contingent on what the adjacent overwritten memory pertains to
(e.g., function pointers, sensitive data).

Summary:
The vulnerability arises from treating memory allocated for one type of object (Type1) as if it were another (Type2), without appropriate checks or validation regarding the size or properties of allocated memory. This results in an Out-of-Bounds write through the option field on a Type1 object improperly casted as Type2. This segment of code thus represents a critical security flaw that needs rectification to prevent potential exploits.git

Academic papers



Detecting Kernel Memory Leaks in Specialized Modules with Ownership Reasoning

Navid Emamdoost
University of Minnesota
navid@cs.umn.edu

Qiushi Wu
University of Minnesota
wu000273@umn.edu

Kangjie Lu
University of Minnesota
kjlu@umn.edu

Stephen McCamant
University of Minnesota
mccamant@cs.umn.edu

Abstract—The kernel space is shared by hardware and all processes, so its memory usage is more limited, and memory is harder to reclaim, compared to user-space memory; as a result, memory leaks in the kernel can easily lead to high-impact denial of service. The problem is particularly critical in long-running servers. Kernel code makes heavy use of dynamic (heap) allocation, and many code modules within the kernel provide their own abstractions for customized memory management. On the other hand, the kernel code involves highly complicated data flow, so it is hard to determine where an object is supposed to be released. Given the complex and critical nature of OS kernels, as well as the heavy specialization, existing methods largely fail at effectively and thoroughly detecting kernel memory leaks.

In this paper, we present K-MELD, a static detection system for kernel memory leaks. K-MELD features multiple new techniques that can automatically identify specialized allocation/deallocation functions and determine the expected memory-release locations. Specifically, we first develop a usage- and structure-aware approach to effectively identify specialized allocation functions, and employ a new rule-mining approach to identify the corresponding deallocation functions. We then develop a new ownership reasoning mechanism that employs enhanced escape analysis and consumer-function analysis to infer expected release locations. By applying K-MELD to the Linux kernel, we confirm its effectiveness: it finds 218 new bugs, with 41 CVEs assigned. Out of those 218 bugs, 115 are in specialized modules.

I. Introduction

even though it will never be used again. A memory region is definitely leaked when all the pointers to the allocated memory go out of scope or are overwritten. Leaked memory becomes unusable until the system reboots. Kernel memory is typically never swapped out, so a kernel memory leak reduces the physical memory available for any other purpose. Due to increased paging, a memory leak can hurt performance [11] and eventually exhaust all the available memory. Kernel memory leaks happen
such paths an
with a memo
memory leak
many small l
same situation
only very ran
user may find

For examp
discovered by
Here the func
memory via
message is st
allocation suc
execution retu
execution con
report_alg
report_alg

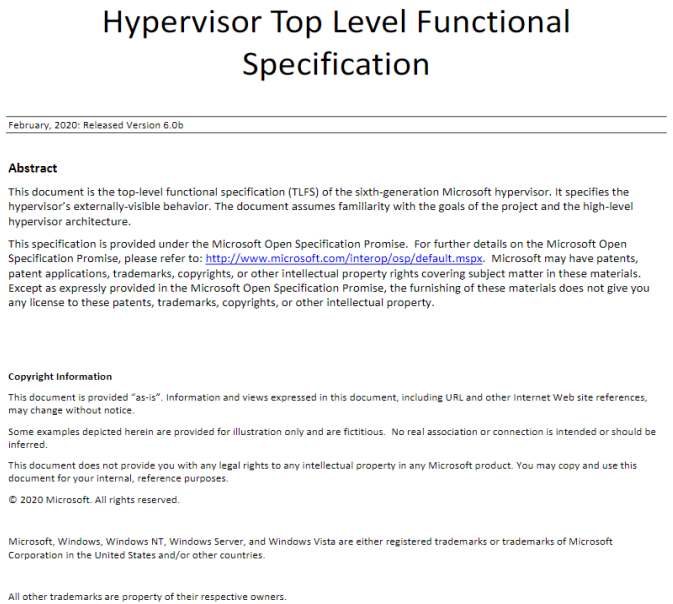
```
/* File: crypto/crypto_user_base.c */
static int crypto_report(struct sk_buff *in_skb,
struct nlmsghdr *in_nlh, struct nlattr **attrs)
{
struct crypto_dump_info info;
...
alg = crypto_alg_match(p, 0);
if (!alg)
return -ENOENT;

err = -ENOMEM;

skb = nlmsg_new(NLMSG_DEFAULT_SIZE, GFP_KERNEL);
if (!skb)
goto drop_alg;
...
info.out_skb = skb;
...
err = crypto_report_alg(alg, &info);

drop_alg:
crypto_mod_put(alg);

if (err)
return err;
...
}
```

```
kernel-sec
Analyzing the provided C code snippet that involves a network interactions function in a cryptographic module,
let's determine if there is a bug:

Memory Allocation: Memory for skb is allocated at line 13 with nlmsg_new.
Check for Allocation Failure: There is a check at line 14 to see if the allocation failed.
If it fails, the code jumps to drop_alg at line 21.
Resource Releasing: At drop_alg, crypto_mod_put(alg) is called to decrease the
reference count of the alg object. This is good practice.
Memory Leak Handling: However, if there's an error after skb has been correctly
allocated and used (the error might be set during the execution of
crypto_report_alg function at line 19), the function returns the error
at line 25 without freeing the allocated skb.
This indicates a potential memory leak scenario.
Based on this analysis, yes, there is a bug in the code,
specifically a memory leak, where skb is not freed when there is an error after
skb has been allocated and used.

This leak can be fixed by adding a line to free skb before returning the error
at line 25
```

THE LINUX FOUNDATION

Fine-tuning--Architecture docs

- Help with analysing kernel hv_* code
- Relatively small sample
- 222 pages
- Lots of arch diagrams
- Lots of tables
- Tokenization issues
- Format conversation
- Multiple model usage
- Found 4 issues in hv_ related dev codes
  - (not tested on mainstream yet)



Hypervisor Top Level Functional Specification

February, 2020: Released Version 6.0b

Abstract

This document is the top-level functional specification (TLFS) of the sixth-generation Microsoft hypervisor. It specifies the hypervisor's externally-visible behavior. The document assumes familiarity with the goals of the project and the high-level hypervisor architecture.

This specification is provided under the Microsoft Open Specification Promise. For further details on the Microsoft Open Specification Promise, please refer to: http://www.microsoft.com/interop/osp/default.mspx. Microsoft may have patents, patent applications, trademarks, copyrights, or other intellectual property rights covering subject matter in these materials. Except as expressly provided in the Microsoft Open Specification Promise, the furnishing of these materials does not give you any license to these patents, trademarks, copyrights, or other intellectual property.

Copyright Information

This document is provided "as-is". Information and views expressed in this document, including URL and other Internet Web site references, may change without notice.

Some examples depicted herein are provided for illustration only and are fictitious. No real association or connection is intended or should be inferred.

This document does not provide you with any legal rights to any intellectual property in any Microsoft product. You may copy and use this document for your internal, reference purposes.

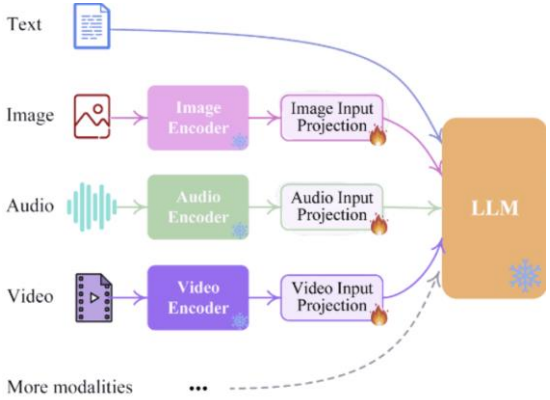
Microsoft, Windows, Windows NT, Windows Server, and Windows Vista are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

All other trademarks are property of their respective owners.

More Challenges

- Evaluation Samples
  - 32 mainstream patches (29 successful match)
  - 15 kernel code files (11 issues found)
- Advanced fine-tuning techniques
  - Multimodal approach
    - Arch images, conf videos, audio description of attacks
- Security considerations

[Mitigations | MITRE ATLAS™]



Thank You!

Your Profile
linkedin.com/in/zahratarkhani

Website
zatkh.github.io/ (Personal)


LINUX
SECURITY
SUMMIT
NORTH AMERICA